AGILITY AND REQUIREMENTS ENGINEERING

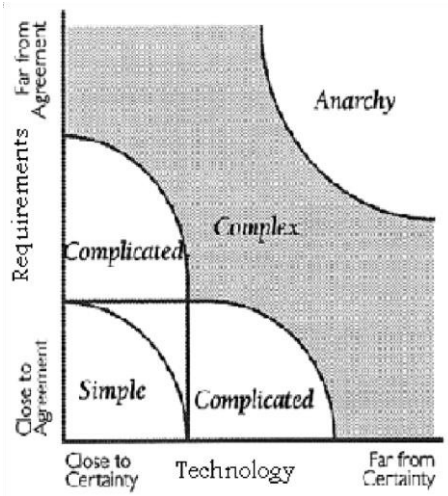
Impact of Agile Processes in RE–Current Agile Practices – Variance – Overview of RE Using Agile – Managing Unstable Requirements – Requirements Elicitation – Agile Requirements Abstraction Model – Requirements Management in Agile Environment, Agile Requirements Prioritization – Agile Requirements Modeling and Generation – Concurrency in Agile Requirements Generation.

Impact of Agile Processes in RE-current agile pratices:

- Current Agile Practices
- Agile Experiences
- Variance Between Formal Requirements Engineering and Agile Processes

1. Current Agile Practices

Building advanced technology, competitive systems is complicated. As shown in the requirements/technology figure1 , the degree of complexity increases as the requirements are less known and the technology is less certain



1.Iterative development – frequent iterations generate increments of work that can be inspected to determine the state of the project and serve as a basis for adaptation.

2. Increments of work - composed of working system functionality rather than artifacts. These increments create a 1:1 relationship between progress and product delivery, and provide a mechanism for user feedback to real product rather than arcane internal artifacts.

3. Collaboration – the customers and engineers form teams that work together.

4. Daily meetings – provide a daily picture of the internal status of a project.

5. Adaptation – the teams of developers selforganize daily based on the daily meetings and the developers and customers self-organize at the end of every increment to guide the project to create the greatest value.

6. Emergence2 – the architecture, team structure, and requirements emerge during the course of the project rather than being determined at its outset.

2. Agile Experiences

At a recent Agile Workshop3, the subject of emergence was discussed. The participants indicated that they had successfully applied emergence in literally hundreds of projects over the last seven years. These projects ranged from relatively straightforward systems to n-tier, complex systems applying cutting edge technology to poorly defined business problems. Business, embedded, and life critical systems had been successfully developed.

The participants felt that the use of refactoring, or periodic cleaning up of business and technical architecture, was required. Otherwise, in all likelihood, the resulting system would be difficult to maintain and sustain. It would contain unrectified designs and code that wasn't orthogonal. However, none of the participants was able to explain why emergence worked.

3. Variance Between Formal Requirements Engineering and Agile Processes

The fundamental principle underlying requirements engineering is the assumption that a system should be clearly specified before its design and implementation can start. Ideally, a specification should be unambiguous, consistent, concise, traceable, implementation independent, etc., and signed off by all stakeholders involved before any further development is done. Failing to do so has in the past caused tremendous cost.

Variance:

- **Variance** represents the comparison of the effort that was planned (Estimated Hours) amongst the actual efforts (Spent Hours) with in a sprint.
- **X Axis** – shows last 15 sprints.
-



1. If the Spent Hours is less than the Estimated Hours, then Effort Variance will be positive.

2. If the Spent Hours is more than the Estimated Hours, then Effort Variance will be negative.If the Variance is positive then the indicator will be in green and spiked up. If the variance is negative then the indicator will be in red and spiked down.

3. Positive Variance represents that the team member has spent less time (in Hours) in comparison of their Estimation (in Hours) within a sprint.

4. Negative Variance represents that the team member has spent more time (in Hours) in comparison of their Estimation (in Hours) within a sprint.

5. **Orange Bar** – Total Hours of work estimated by the Team members (in Hours) with in a sprint.
6. **Green Bar** – Total Hours of time spent by a Team members (in Hours) with in a sprint.
7. **Green Trend line** – It will show total available hours of all the sprints within a sprint.
8. **Variance** is based on total hours that are spent more/ less in comparison of your estimated hours.
(Sprint Effort Variance = Estimation Hours – Spent Hours).

Overview of RE using agile managing unstable requirements:

Agile development is a better way of managing the changing requirements that many software development projects face. There's been a lot of thought around Agile change management. Here are five ways Agile helps manage changing requirements:

- Customer input happens throughout the development process
- Product backlog sets development priorities
- Daily meetings promote communications
- Task boards make developer tasks and details visible
- User stories and sprints orchestrate change

1. Customer input happens throughout the development process

Back when I got my start as a technical writer, I worked on software development projects that lasted months. The software wasn't completed until the "end of the project."

- Gains customer and stakeholder feedback on features sooner rather than later
- Improves scope control because stakeholders can add new requirements, shift priorities, or rethink requirements on a feature or architectural level
- Gives project teams the room to take risks and innovate based on customer feedback without sacrificing too much time or budget because agile teams can pivot on requirements as needed

2. Product backlog sets development priorities

The one element that took me a while to warm up to was the product backlog. It's like the scrum team's prioritized to-do list of features to build.

Managing or grooming the product backlog can be an art unto itself. In some organizations, the scrum master manages the backlog. Other organizations might choose to have product managers or cross-functional team leaders involved in managing it.

3. Daily meetings promote communications

Holding daily meetings, or stand-ups, are another tool for managing changing requirements. These meetings take place at the same time each day and give team members a chance to talk about the tasks they've completed and any obstacles standing in their way.

A properly managed daily meeting lets developers, team leads and stakeholders (if invited) to share information. Some of that information could be issues and feedback about product requirements that might arise during the implementation process.

4. Task boards make developer tasks and details visible

Product requirements documents are too often read once and left in an email inbox for the duration of the project. Agile development uses the concept of a task board to divide up tasks into multiple columns and make them visible 24/7. These boards parcel out projects into the following stages:

- To do
- In progress
- In testing
- Done

Tasks boards help manage changing requirements because of the visibility they offer, which include:

- Project requirement status is visible to every team member.
- Dependencies of project requirements impacted by changing requirements are clear.
- Shows threaded comments about the changing requirements before and during sprints from the developer and other team members.

5. User stories and sprints orchestrate change

I spent some of my formative years in the IT industry writing requirement documents. I became accustomed to documenting every feature that was being built. When I was first learning about Agile, I came to marvel at how, if you orchestrated user stories and sprints appropriately, you can manage change quite effectively.

For example, a product owner creates a story. Developers can build a new application feature based on the story. During or after the sprint where that feature is built, a salesperson delivers feedback from a customer that shows the feature is missing critical functionality. The product owner can create a new story to build out the feature with the missing functionality during the next sprint.

Requirements Elicitation:

Requirements elicitation is the process of gathering and defining the requirements for a software system. The goal of requirements elicitation is to ensure that the software development process is based on a clear and comprehensive understanding of the customer needs and requirements. Requirements elicitation involves the identification, collection, analysis, and refinement of the requirements for a software system. It is a critical part of the software development life cycle and is typically performed at the beginning of the project. Requirements elicitation involves stakeholders from different areas of the organization, including business owners, end-users, and technical experts. The output of the requirements elicitation process is a set of clear, concise, and well-defined requirements that serve as the basis for the design and development of the software system.

Requirements elicitation is perhaps the most difficult, most error-prone and most communication intensive software development. It can be successful only through an effective customer-developer partnership. It is needed to know what the users really need.

Requirements elicitation Activities:

Requirements elicitation includes the subsequent activities. Few of them are listed below –

- Knowledge of the overall area where the systems is applied.
- The details of the precise customer problem where the system are going to be applied must be understood.
- Interaction of system with external requirements.
- Detailed investigation of user needs.
- Define the constraints for system development.

Requirements elicitation Methods:

There are a number of requirements elicitation methods. Few of them are listed below –

1. Interviews
2. Brainstorming Sessions
3. Facilitated Application Specification Technique (FAST)
4. Quality Function Deployment (QFD)
5. Use Case Approach

The success of an elicitation technique used depends on the maturity of the analyst, developers, users, and the customer involved.

1.Interviews:

Objective of conducting an interview is to understand the customer's expectations from the software.

It is impossible to interview every stakeholder hence representatives from groups are selected based on their expertise and credibility.

Interviews maybe be open-ended or structured.

1. In open-ended interviews there is no pre-set agenda. Context free questions may be asked to understand the problem.
2. In structured interview, agenda of fairly open questions is prepared. Sometimes a proper questionnaire is designed for the interview.

2. Brainstorming Sessions:

- It is a group technique
- It is intended to generate lots of new ideas hence providing a platform to share views
- A highly trained facilitator is required to handle group bias and group conflicts.
- Every idea is documented so that everyone can see it.
- Finally, a document is prepared which consists of the list of requirements and their priority if possible.

3. Facilitated Application Specification Technique:

It's objective is to bridge the expectation gap – difference between what the developers think they are supposed to build and what customers think they are going to get.

A team oriented approach is developed for requirements gathering.

Each attendee is asked to make a list of objects that are-

1. Part of the environment that surrounds the system
2. Produced by the system
3. Used by the system

Each participant prepares his/her list, different lists are then combined, redundant entries are eliminated, team is divided into smaller sub-teams to develop mini-specifications and finally a draft of specifications is written down using all the inputs from the meeting.

4. Quality Function Deployment:

In this technique customer satisfaction is of prime concern, hence it emphasizes on the requirements which are valuable to the customer.

3 types of requirements are identified

- **Normal requirements:** In this the objective and goals of the proposed software are discussed with the customer. Example – normal requirements for a result management system may be entry of marks, calculation of results, etc
- **Expected requirements:** These requirements are so obvious that the customer need not explicitly state them. Example – protection from unauthorized access.
- **Exciting requirements:** It includes features that are beyond customer's expectations and prove to be very satisfying when present. Example – when unauthorized access is detected, it should backup and shutdown all processes.

The major steps involved in this procedure are –

1. Identify all the stakeholders, eg. Users, developers, customers etc
2. List out all requirements from customer.
3. A value indicating degree of importance is assigned to each requirement.
4. In the end the final list of requirements is categorized as –
    - It is possible to achieve
    - It should be deferred and the reason for it
    - It is impossible to achieve and should be dropped off

5. Use Case Approach:

This technique combines text and pictures to provide a better understanding of the requirements.

The use cases describe the 'what', of a system and not 'how'. Hence, they only give a functional view of the system.

The components of the use case design includes three major things – Actor, Use cases, use case diagram.

1. **Actor** –
   It is the external agent that lies outside the system but interacts with it in some way. An actor maybe a person, machine etc. It is represented as a stick figure. Actors can be primary actors or secondary actors.
   - Primary actors – It requires assistance from the system to achieve a goal.
   - Secondary actor – It is an actor from which the system needs assistance.
2. **Use cases:**
   They describe the sequence of interactions between actors and the system. They capture who(actors) do what(interaction) with the system. A complete set of use cases specifies all possible ways to use the system.
3. **Use case diagram:**
   A use case diagram graphically represents what happens when an actor interacts with a system. It captures the functional aspect of the system.
   - A stick figure is used to represent an actor.
   - An oval is used to represent a use case.
   - A line is used to represent a relationship between an actor and a use case.

ADVANTAGES OR DISADVANTAGES:

Advantages of Requirements Elicitation:

- Helps to clarify and refine the customer requirements.
- Improves communication and collaboration between stakeholders.
- Increases the chances of developing a software system that meets the customer needs.

Disadvantages of Requirements Elicitation:

- Can be time-consuming and expensive.
- Requires specialized skills and expertise.
- May be impacted by changing business needs and requirements.

Agile Requirements Abstraction Model:

This article presents an investigation of the possibility to integrate traditional requirements engineering in an agile software development method. One reason for integrating a traditional with an agile method was that the case company experienced difficulties with the agile requirements engineering process. The results are based on a qualitative case study of one company that uses scrum for its software development. An agile model, with four abstraction levels was developed in close collaboration with industry

Abstraction levels:

Requirements engineering (RE), on the other hand is a traditional method with the goal to identify, analyze, document and validate requirements. Is it possible to bridge the traditional requirements engineering process with agile methods, and still stay agile? According to, requirements engineering and agile methods are seen as incompatible. One reason is that RE often relies on documentation while agile methods are focusing on faceto-face communication between customers and developers. However, several studies have looked into bridging traditional and agile methods. In, the study shows that it is possible to successfully integrate a traditional project management approach with XP projects. Furthermore, Boehm and Turner concluded that agile concepts will continue to integrate into traditional organizations and vice versa. In addition, Nawrocki et al. proposed modifications to the XP methodology to embrace RE practices. One of the modifications was to have a RE phase in the beginning of the project that provides a wider perspective, and still manage to stay as lightweight as the original XP.

ARAM STRUCTURE

This section describes the ARAM's structure. The version of ARAM presented in this paper is based on the development of ARAM at Tactel and is an example of how ARAM can look like. The concepts of the ARAM model as described in this paper, makes it possible for any organization to adapt and tailor ARAM to fit their specific organization. The aim of ARAM is to bridge the traditional approaches and agile methods by allowing project plans and contacts, and still keep time-boxed iterations where the development team can work on their sprint backlog items. ARAM does not interfere with the scrum sprint, rather focus on making the pre-project requirements engineering efficient.

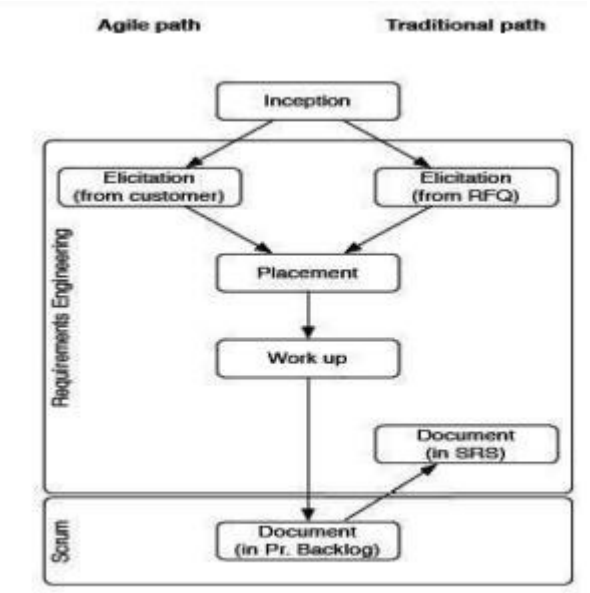


Figure 1. ARAM structure

Project Inception Phase

In a contract-driven business scenario, the customer contacts the software vendor and requests for product information. The response to the customer at this stage is an overview of the product containing the product's feature and what value the product offers. The agile information should include an overview of scrum, target-cost contracts and how the agile path works. The contract and requirements processes will ultimately end up in the product backlog, but there are two different paths to reach that goal. If market-driven development process is preferred, there is no direct customer to contact. In this case, the project inception may come from the product manager, marketing department, or from an innovation meeting at the producing company where high level goals are derived. In the agile path, a target-cost contract with a variable scope is selected. The delivery date and the budget are known in advanced while requirements will be implemented based on their priorities.

Scrum and ARAM

Requirements in the product backlog need to be prioritized according to most value for the customer. One main advantage by using ARAM is that all requirements can be compared. A requirement on feature level should only be compared with other requirements on the feature level. In other words, requirements should only be compared with requirements on the same abstraction level. After prioritizing all requirements in the product backlog, they are analyzed at the next sprint planning meeting. The high priority requirements are selected for the coming sprint. When estimating the effort for each story, requirements need to be broken down to at least the functional abstraction level to be able to make as accurate estimations as possible. An agile practice and basic rule of scrum, is to develop a common language for all team members which was pointed out as one problem in the case study . Therefore, it is desirable to have common names for requirements at each abstraction level, as is illustrated in figure.

To get structure from this set of requirements RAM specifies a number of abstraction levels. In the study by Gorschek the number of abstraction levels are four, but they state that this is tailored for their specific case study and can be variable. When implementing for a specific organisation, more or fewer levels can be used. The four level model can be seen in figure.

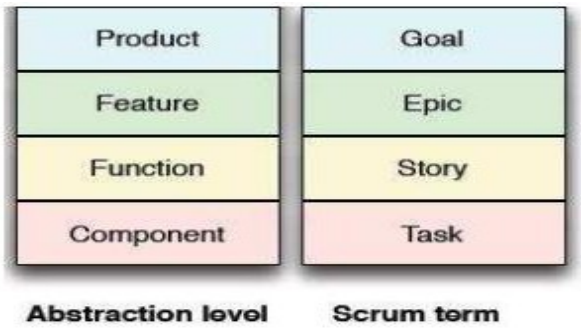
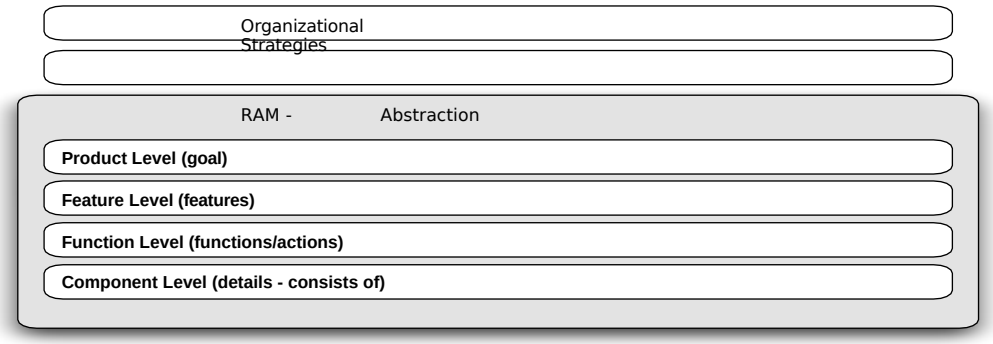
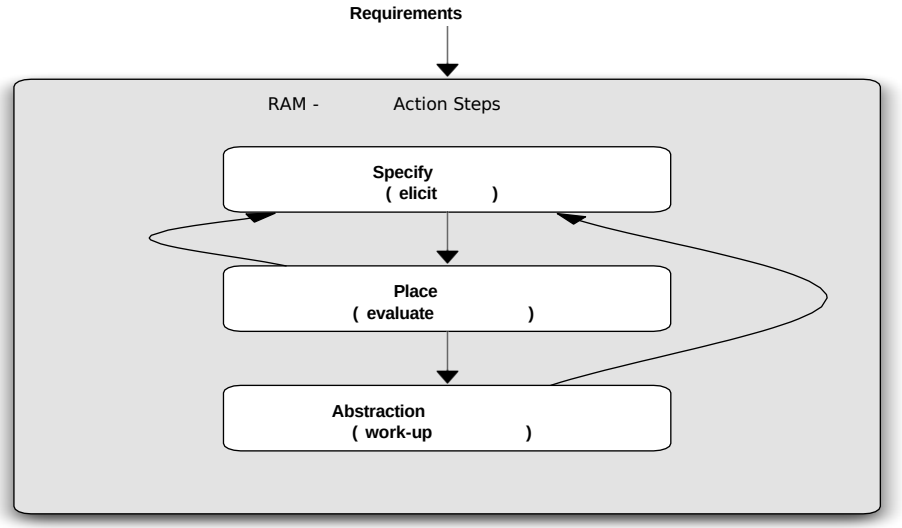


Figure 6. Scrum terms and abstraction levels

The Product level represents the most abstract requirements.



Action steps : RAM has three specified action steps (figure 11) to achieve the right requirements specification based on the abstraction levels. These steps are iterated for all requirements in the system. First of all, the requirement should be specified (elicited). This is the first sketch of an idea about what should be included in the system. Here the four most general attributes to the requirement - including [32] description, reason/benefit/rationale, restrictions/risks and title - are specified. After this specification the requirement is placed (evaluated) according to the heart of RAM model



Attributes

In the process of specifying, placing and working-up the requirements, additional attributes are added. These include the two groups traceability and process attributes for fitting into RAM. The traceability attributes suggested by RAM are *Requirement source*, *Requirement owner*, *Requirements manager* and *Relation/dependency*. The process attributes include *State*, *Reject reason (for rejected requirements only)*, *Due date*, *Version*, *Date of creation* and *Last changed*. Many of the attributes are the same or almost the same as in traditional RE with some anomalies

Requirements management in agile environment

In many ways, the manner of capturing requirements in an Agile project management environment is similar to a *"waterfall,"* or traditional project management environment - numerous meetings with subject matter experts, end users, walkthrough / documenting the current business workflow, creating mockups, etc. However, Agile and traditional project management approaches contrast in how requirements are managed over time.

Some fundamental differences in managing requirements include:

Traditional Requirements Management	Agile Requirements Management
In a "waterfall" project management environment, the approach is to capture and define **all** of the end-state requirements upfront.	In an Agile project management environment, while high-level requirements are also captured upfront, it is understood that requirements may evolve over the course of the effort.
Requirements are documented in a business requirements document (BRD) or business specifications document (BSD) for the purpose of designing the end state of a product.	The effort begins with a high-level scope agreement where initial business requirements are translated into user stories. Those user stories specify the needs of the product based on the information at the time given.
The goal is to understand the "as-is" state of the existing product or the business gaps that define the lack, so that the "to-be" state of the desired product can be defined.	The goal is no longer focused on eliciting the "as-is" in order to the define the "to-be," but to clarify and ensure understanding of the business need for all users.
By defining the end-state of the application from the beginning, there is little room for change once development begins.	As more information becomes known over time, the team is better able to adjust and make changes accordingly.

Managing requirements in an Agile project management environment is to think through the full life cycle of the requirement; to consider the full user experience and even beyond the defined stakeholders

Agile Requirement Prioritization:

Prioritization in literary terms means the decision of arranging things in order of their importance. Prioritization in agile is the act of deciding in what order the agile team will work on the requirements in a project

Here is the list of popular prioritization techniques:

- MoSCoW prioritization
- Kano model
- Relative weighting method
- Opportunity Scoring
- Stack Ranking
- Priority Poker
- Cost of Delay
- 100 Dollar Test

Agile Prioritization Factors

- The financial value of the requirements is a major factor to be considered in prioritizing requirements. The value could be expressed as new revenue, incremental revenue, or as operational efficiency.
- The cost of developing the requirements is another essential factor to be considered by the product owner. Value and cost together indicate the RoI for the requirements.
- The next factor to be considered in prioritization is the amount and significance of knowledge and capabilities that the team will gain while working on the requirements.
- Understanding the level of risks involved in introducing the new features is essential in the process of prioritization.

Popular Prioritization Techniques

- MoSCoW prioritization – popularized by the DSDM methodology.
- Kano model – introduced by Prof. Noriaki Kano
- The relative weighting method – by Karl Wiegers

8 Popular Prioritization Techniques

1. MoSCoW prioritization – popularized by the DSDM methodology.
2. Kano model – introduced by Prof. Noriaki Kano
3. The relative weighting method – by Karl Wiegers

4. Opportunity Scoring
5. Stack Ranking
6. Priority Poker
7. Cost of Delay
8. 100 Dollar Test

MoSCoW Agile Prioritization Technique

MoSCoW Prioritization in Agile: In the DSDM methodology, the priorities are expressed as per the MoSCoW model:

- Must– The must requirements is given the topmost priority
- Should– Next priority is given to the requirements that are highly desirable, though not mandatory
- Could– The next priority is given to the requirement that is nice to have
- Won't– And the final consideration is given to the requirements which will not work in the process at that point of time.

Kano Model of Prioritization in Agile

Professor Noriaki Kano propagated Kano Model of Prioritization. This prioritization technique involves three levels that include considering customer satisfaction from disappointment to not happy to immediate happiness to get delighted. Two important factors that create an impact on the satisfaction level during this prioritization are the existence of features and the degree of implementation.

Relative Weighting Prioritization Technique

The relative weighting scheme is a simple model where prioritization is done based upon all the factors mentioned above. The major factors considered in relative weighing prioritization technique are:

- The value of a feature and the negative impact that might be caused by the absence of the feature
- Based on the expert judgment made by the product owner and supported by the agile team in ranking the score of features in the following way (a scoreboard from 1 to 9 is usually used)

- Benefit from having the feature
- Penalty for not having the feature
- Cost of producing the feature
- The risk incurred in producing the feature

Opportunity Scoring

Opportunity Scoring is a beneficial prioritization method used by organizations to develop agile products. This prioritization model uses data from market research to help determine what the users expect from your product or service. It allows organizations to create the schedule according to their target audience's wants and needs.

Stack Ranking

Stack Ranking is one of the most popular forms of prioritization techniques that is currently used by a lot of software companies. It is also one of the easiest techniques that allow prioritization based on the user story.

Priority Poker

This agile priority technique is based on similar rules as actual poker played with cards. When playing poker, prioritization is done in a calculative manner, with big wins being the ultimate goal. Similarly, in agile priority poker, items that will yield the highest results in specific target markets are given priority.

Cost of Delay

The objective of this prioritization technique is to understand how much money would the organization lose if a certain feature is not available. This prioritization focuses on monetary loss to understand which features are the most important and the list is created accordingly. It is a proactive approach to ensure the manager fight fires and deal with emergencies that can result in losses.

100 Dollar Test

This technique is also known as Cumulative Voting and is a straightforward process. It is similar to the poker technique but each stakeholder is given 100 points or dollars to assign to each feature or task.

The stakeholders divide their 100 dollars by assigning a spending amount to each feature. Once all the 100 dollars are spent, the moderator then tallies all the points and the feature with the most dollars assigned is given the highest priority, followed by tasks with the next highest amounts.

Agile Requirements Modeling and Generation

The basic idea of agile requirements modeling is to create a foundation for your project's specifications for higher-level understanding. This is done in the beginning stages with details being added as you need them on a just-in-time basis.To fully understand what this model is, it's important to look at agile software development too.

The best way to understand this process is to look at its main goals. Agile requirements modeling is designed to support the goals of software development and aims to achieve the following objectives:

- Establish the best practices for an effective model
- Outline the ways to put those practices into place
- Display alternatives to improve the modeling approach

Types of Requirements:

There are two categories that agile requirements modeling can be divided into. The distinction between the two is important to establish as they distinguish the technicalities of a project from the user interactions.

These two categories are:

1. Behavioral: This form of requirement refers to any user interface issues, usage, and business rules. This can also be seen as the functional requirements of a project.
2. Non-behavioral: On the technical side, non- behavioral looks at a system's features relating to availability, security, performance, interoperability, dependability, and reliability. In other words, these are the non- functional requirements.

Applying Agile Requirements Modeling Principles

The future of your agile project relies heavily on several factors. As any developer already knows, anything could happen, from client changes to user-need adaptations, that can turn the project's success into failure.

Without a clear plan or project outline, a system can quickly become confused by varying stories and goals. However, agile requirements modeling prevents this from happening and ensures that the same goal is communicated at all times.

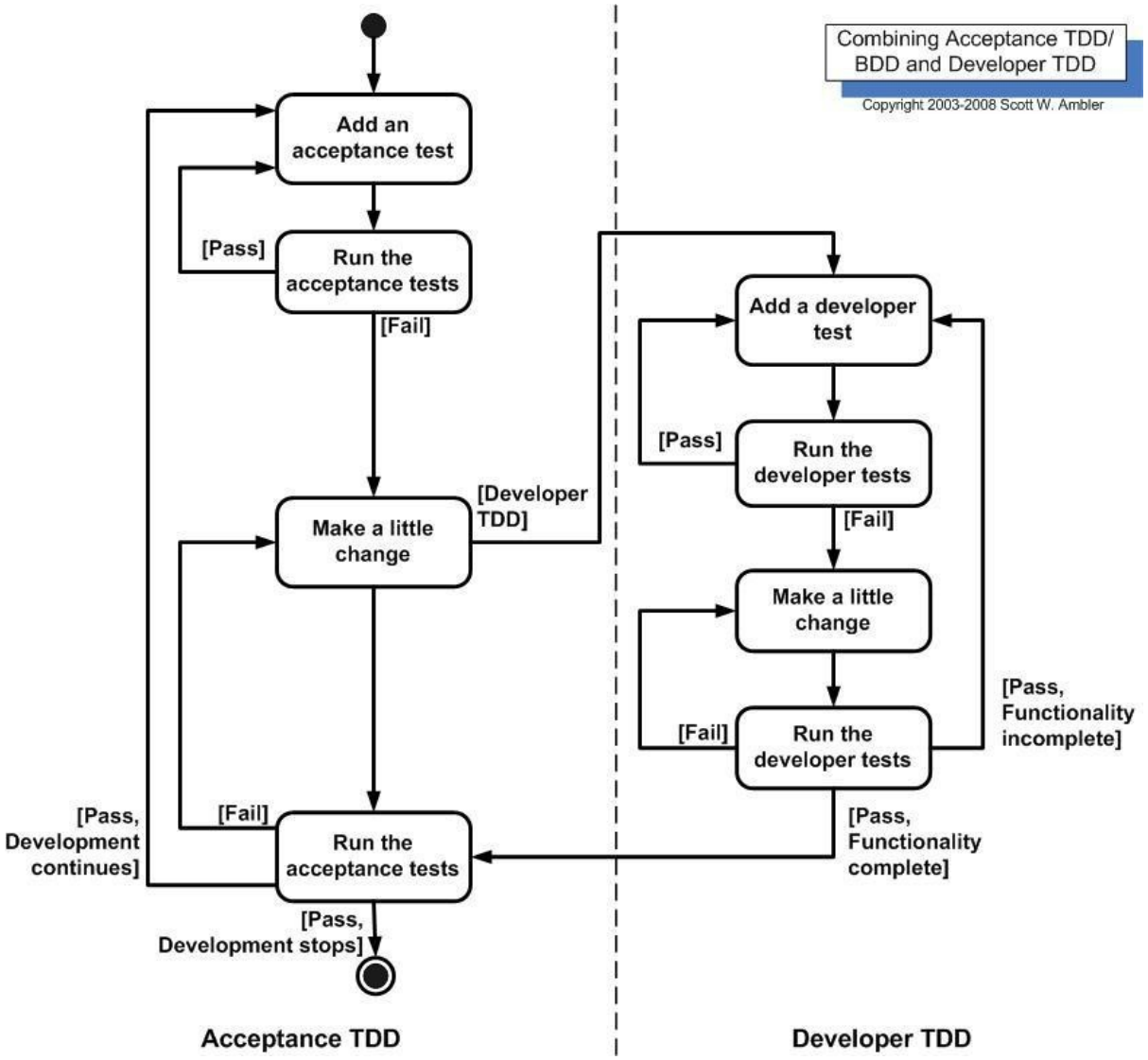
With this in mind, when it comes to agile modeling, getting the key principles right is crucial. Although this isn't a rigid procedure, keeping these practices at the foundation of your work will steer your project towards a more streamlined process.

These principles include:

- Outlining how the software is your primary goal
- Establishing the next effort as your secondary goal
- Remaining agile by focusing only on the models required
- Assuming simplicity
- Embracing change
- Modeling with purpose
- Creating multiple models
- Producing quality work
- Maximizing stakeholder investment

The key benefits of this modeling form are based on the corporation of the above principles. Agile modeling is a synergistic approach to development that requires all of these practices to be utilized. Leaving one out can open up the project to becoming less effective.

The reason for this is that software is complex and it needs to be analyzed from various standpoint.



Concurrency in Agile Requirements Generation:

The major activities required in a software development life cycle (SDLC) were identified in the waterfall model. The agile concurrent software process model proposes that the activities identified in the waterfall model are not done sequentially; these activities progress concurrently with varying intensities during the entire software life cycle. Also, a focus on a working software subset as opposed to documents, giving high importance to customer satisfaction and sensitivity to human factors go a long way in contributing towards a software project's overall success.

WATERFALL MODEL

It was understood that software had a life-cycle starting from the decision to make that software to the time the software became operational. The term software development life-cycle was based on a basic software process model, indicating the stages software would go through in its life-cycle. The first response to the software crisis was the discovery of the waterfall software process model. It was thought that the root cause of the problem was that the requirements were not understood well enough before the software design and proper design was not done prior to coding.

Applying the engineering paradigm of freezing requirements before design and freezing design before manufacture, the waterfall model stipulated that software should be made in a sequential series of stages, like requirements' analysis, design, detailed design, coding, integration, installation, operations and maintenance. Slowly, it became apparent that it was not possible to have water-tight compartments and it was desirable to have feedback between successive stages. Thus a design error found during coding would lead to a correction of design and then the coding would continue. This was in line with the engineering practice for a design error found during manufacture is first corrected in design and than manufacturing is done as per the modified design. Also, taking the clue from engineering again, it was felt that making a prototype of the system in early stages would help in understanding the requirements better and would eventually help in making a better system.

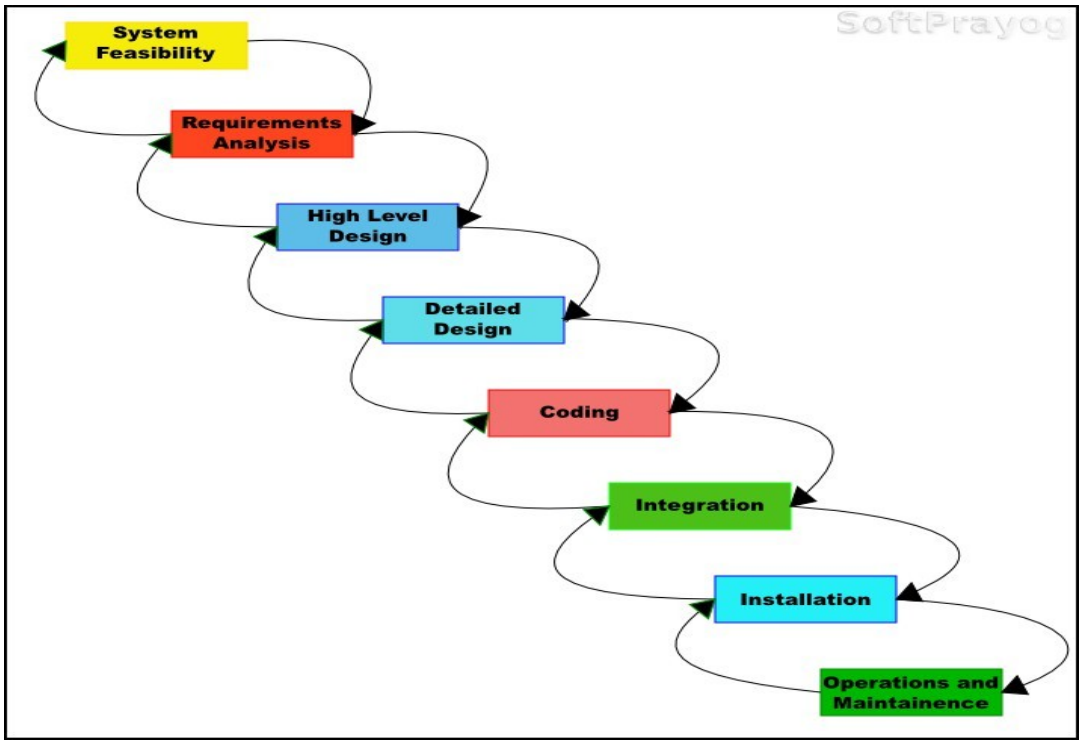


Figure 1: The Waterfall Software Process model